

	INSTITUTO FEDERAL DA PARAÍBA - IFPB Campus Campina Grande Curso: Engenharia da Computação Componente: Internet das Coisas Professor: Alexandre Sales Vasconcelos Estudante(s): Sophia Sales Fernandes, Maria Clara Colaço da Costa e Pedro Augusto Gonçalves Lucena
---	---

Projeto Final de IoT: Sistema de Iluminação Inteligente com Zigbee, BBB e Interface Web

1. Preparação do cartão SD

Inicialmente, foi necessário preparar novamente o cartão microSD com a imagem Linux embarcada para a BeagleBone Black. Para isso, a imagem foi gravada no cartão utilizando o BBB Imager.

Após a gravação, o microSD foi inserido na BBB e o sistema foi inicializado pelo cartão SD. Com isso, a placa ficou pronta para receber as configurações necessárias do projeto final de iluminação inteligente.

2. Inicialização da BeagleBone Black

Após a inicialização do sistema Linux embarcado, a BBB foi conectada à rede local por cabo Ethernet, permitindo o acesso à placa pela rede e a execução da aplicação web do projeto.

Em seguida, foi realizado o acesso remoto à placa via SSH para continuar a configuração do sistema, instalar os pacotes necessários e preparar a comunicação com o Node B.

3. Instalação dos pacotes necessários

Com acesso ao terminal Linux da BBB, foi realizada a atualização dos pacotes do sistema:

```
sudo apt update
```

Depois disso, foram instalados o SQLite e os pacotes necessários para uso do Python:

```
sudo apt install python3-pip python3-venv sqlite3 -y
```

Em seguida, foi criado um ambiente virtual Python para organizar as dependências da aplicação web:

```
python3 -m venv venv  
source venv/bin/activate
```

Dentro do ambiente virtual, foram instaladas as bibliotecas utilizadas no projeto:

```
pip install flask pyserial
```

Esses passos foram necessários para preparar a BBB para executar a interface web, armazenar os eventos no banco SQLite e realizar a comunicação serial UART com o Node B.

4. Configuração da comunicação UART

No projeto final, a comunicação entre a BeagleBone Black e o Node B foi feita por meio de UART, utilizando os pinos TX, RX e GND. Essa comunicação foi necessária para que a interface web executada na BBB pudesse enviar comandos diretamente ao Node B e também receber atualizações sobre o estado da iluminação.

A ligação foi feita da seguinte forma:

```
BBB TX → RX do Node B  
BBB RX → TX do Node B  
GND da BBB → GND do Node B
```

Na BBB, foi utilizada a porta serial:

```
/dev/ttyS1
```

No código Python da aplicação web, essa porta foi configurada juntamente com a taxa de comunicação:

```
SERIAL_PORT = "/dev/ttyS1"  
BAUD_RATE = 115200
```

Para permitir a comunicação serial, também foi necessário realizar ajustes na configuração da UART da BBB, habilitando a porta utilizada pela aplicação.

O arquivo alterado foi:

```
sudo nano /boot/uEnv.txt
```

Após a alteração, a BBB foi reiniciada para que a configuração fosse aplicada. Com isso, a aplicação Python conseguiu se comunicar com o Node B pela UART.

5. Verificação da comunicação UART

Após a configuração da UART, foram realizados testes para verificar se a BBB conseguia se comunicar corretamente com o Node B.

Primeiramente, foi testada a abertura da porta serial utilizada no projeto:

```
/dev/ttyS1
```

Em seguida, a aplicação Python foi executada na BBB. Ao acessar a interface web e clicar nos botões de controle, a aplicação passou a enviar comandos para o Node B pela UART, como:

CMD:ON
CMD:OFF
MODE:AUTO
MODE:MANUAL

No terminal da BBB, foi possível observar o envio dos comandos, por exemplo:

Enviado para Node B: CMD:ON
Enviado para Node B: CMD:OFF

Também foram recebidas mensagens enviadas pelo Node B informando o estado atual da iluminação e o modo de operação:

STATUS:ON
STATUS:OFF
MODE:AUTO
MODE:MANUAL

Essa verificação confirmou que a comunicação entre a aplicação web na BBB e o Node B estava funcionando, permitindo tanto o envio de comandos da interface quanto o recebimento de atualizações do sistema.

6. Atualização dos Nodes ESP32

No projeto final, os nodes ESP32-C6 foram ajustados para utilizar comunicação Zigbee entre si, substituindo a comunicação Wi-Fi/MQTT utilizada nas etapas anteriores.

O Node A ficou responsável por representar a entrada automática do sistema. Inicialmente a lógica foi pensada para um sensor de presença, mas, por direcionamento do professor, foi utilizado um botão como entrada digital para simular essa detecção. Dessa forma, quando o botão está pressionado, o sistema interpreta como presença detectada e o Node A envia um comando de acionamento para o Node B via Zigbee. Quando o botão é solto, o sistema interpreta como ausência de presença e envia um comando para desligamento.

O Node B ficou responsável por receber os comandos Zigbee enviados pelo Node A, controlar o LED e se comunicar com a BeagleBone Black por meio da UART.

A comunicação entre os nodes passou a seguir a lógica:

Node A → Zigbee → Node B

Enquanto a comunicação entre o Node B e a BBB passou a ser:

Node B ↔ UART ↔ BBB

Além disso, o Node B foi ajustado para trabalhar com dois modos de operação:

- **Modo automático:** o LED é controlado pelas mensagens recebidas do Node A via Zigbee. Nesse modo, comandos de ligar e desligar enviados pela BBB são ignorados;
- **Modo manual:** o LED é controlado pelos comandos enviados pela interface web da BBB. Nesse modo, os comandos automáticos vindos do Node A são ignorados.

Também foi implementado um comportamento semelhante ao de um sensor de presença real. Quando o Node A envia o comando para ligar, o LED é acionado imediatamente. Quando o Node A envia o comando para desligar, o Node B aguarda 5 segundos antes de apagar o LED. Caso durante esse intervalo o Node A envie novamente um comando para ligar, o desligamento é cancelado e o LED permanece aceso.

Dessa forma, o Node B passou a funcionar como o ponto central entre a rede Zigbee e a aplicação web, recebendo comandos do Node A e da BBB, acionando o LED e informando o estado atual da iluminação.

7. Persistência dos eventos no SQLite

Para registrar o histórico de funcionamento do sistema, foi criado um banco de dados SQLite na BBB. Esse banco foi utilizado para armazenar os eventos gerados pela interface web e as atualizações recebidas do Node B pela comunicação UART.

Primeiramente, foi criado o banco de dados:

```
sqlite3 iluminacao.db
```

Dentro do SQLite, foi criada a tabela utilizada pelo projeto:

```
CREATE TABLE eventos (  
id INTEGER PRIMARY KEY AUTOINCREMENT,  
tipo TEXT NOT NULL,  
valor TEXT NOT NULL,  
data_hora TEXT NOT NULL  
);
```

Essa tabela foi utilizada para armazenar:

- o tipo do evento, como **STATUS**, **MODE** ou **WEB_CMD**;
- o valor associado ao evento, como **ON**, **OFF**, **AUTO** ou **MANUAL**;
- a data e hora em que o evento ocorreu.

Posteriormente, o banco passou a ser utilizado pela aplicação web em Python, registrando automaticamente as mudanças de estado da lâmpada, as trocas de modo de operação e os comandos enviados pela interface.

8. Desenvolvimento da aplicação web app.py

Foi desenvolvida uma aplicação web em Python chamada *app.py*, responsável por integrar a interface de controle, o banco SQLite e a comunicação UART com o Node B.

A aplicação foi criada utilizando o framework Flask e ficou acessível pela rede local através do navegador. Com isso, foi possível controlar e monitorar a iluminação a partir de uma página web hospedada na própria BBB.

Na interface web foram implementadas as seguintes funções:

- visualizar o estado atual da lâmpada;
- alternar entre modo automático e modo manual;
- ligar e desligar a lâmpada no modo manual;
- visualizar os últimos eventos registrados;
- acessar uma página com o banco de dados completo.

A comunicação entre a aplicação web e o Node B foi feita pela UART. Quando o usuário interagia com a interface, a BBB enviava comandos para o Node B, como:

CMD:ON
CMD:OFF
MODE:AUTO
MODE:MANUAL

Além disso, a aplicação também recebia mensagens do Node B informando o estado atual do sistema, como:

STATUS:ON
STATUS:OFF
MODE:AUTO
MODE:MANUAL

Dessa forma, a aplicação web passou a ser responsável por centralizar o controle, o monitoramento e o registro dos eventos da iluminação inteligente.

A página principal da aplicação permitiu visualizar o estado atual da lâmpada, o modo de operação, os botões de controle da iluminação e os últimos eventos registrados no banco de dados.

Além disso, a interface foi configurada para atualizar automaticamente em intervalos curtos, permitindo acompanhar as mudanças de estado sem a necessidade de atualizar a página manualmente.

9. Visualização dos registros do sistema

Além da página principal de controle, foi desenvolvida uma página específica para visualizar os dados armazenados no banco SQLite.

Essa página ficou acessível pela própria interface web e permitiu consultar todos os registros gerados pelo sistema durante o funcionamento.

Foram disponibilizadas as seguintes informações:

- tipo do evento registrado;
- valor associado ao evento;
- data e hora de cada ocorrência.

A aplicação também passou a exibir os últimos eventos diretamente na página principal, facilitando o acompanhamento do funcionamento do sistema em tempo real.

Além disso, foi criada uma rota específica para visualizar todos os registros armazenados no banco de dados:

/banco

Com isso, foi possível acompanhar o histórico completo das alterações de estado da iluminação, das mudanças de modo de operação e dos comandos enviados pela interface web.

10. Conclusão

Com o desenvolvimento do projeto final, o sistema de iluminação inteligente passou a integrar comunicação Zigbee, controle via interface web, comunicação UART entre a BBB e o Node B, além do registro local dos eventos em banco SQLite.

A BeagleBone Black ficou responsável por executar a aplicação web, armazenar os dados do sistema e enviar comandos ao Node B. Já os ESP32-C6 ficaram responsáveis pela comunicação Zigbee e pelo acionamento da iluminação.

Além disso, a implementação dos modos automático e manual permitiu separar o controle feito pela entrada automática simulada pelo botão do controle feito pela interface web.

Com isso, o projeto evoluiu para uma arquitetura IoT mais completa, envolvendo dispositivos embarcados, comunicação sem fio Zigbee, comunicação serial UART, aplicação web local e persistência de dados.